

Escola Superior de Tecnologia e Gestão de Águeda

Sistemas Operativos

Técnico Superior Profissional em Redes e Sistemas Informáticos

Comunicação Entre Processos (IPC)

Professor: Ivan Miguel Pires

2025/2026

OBJETIVOS DE APRENDIZAGEM

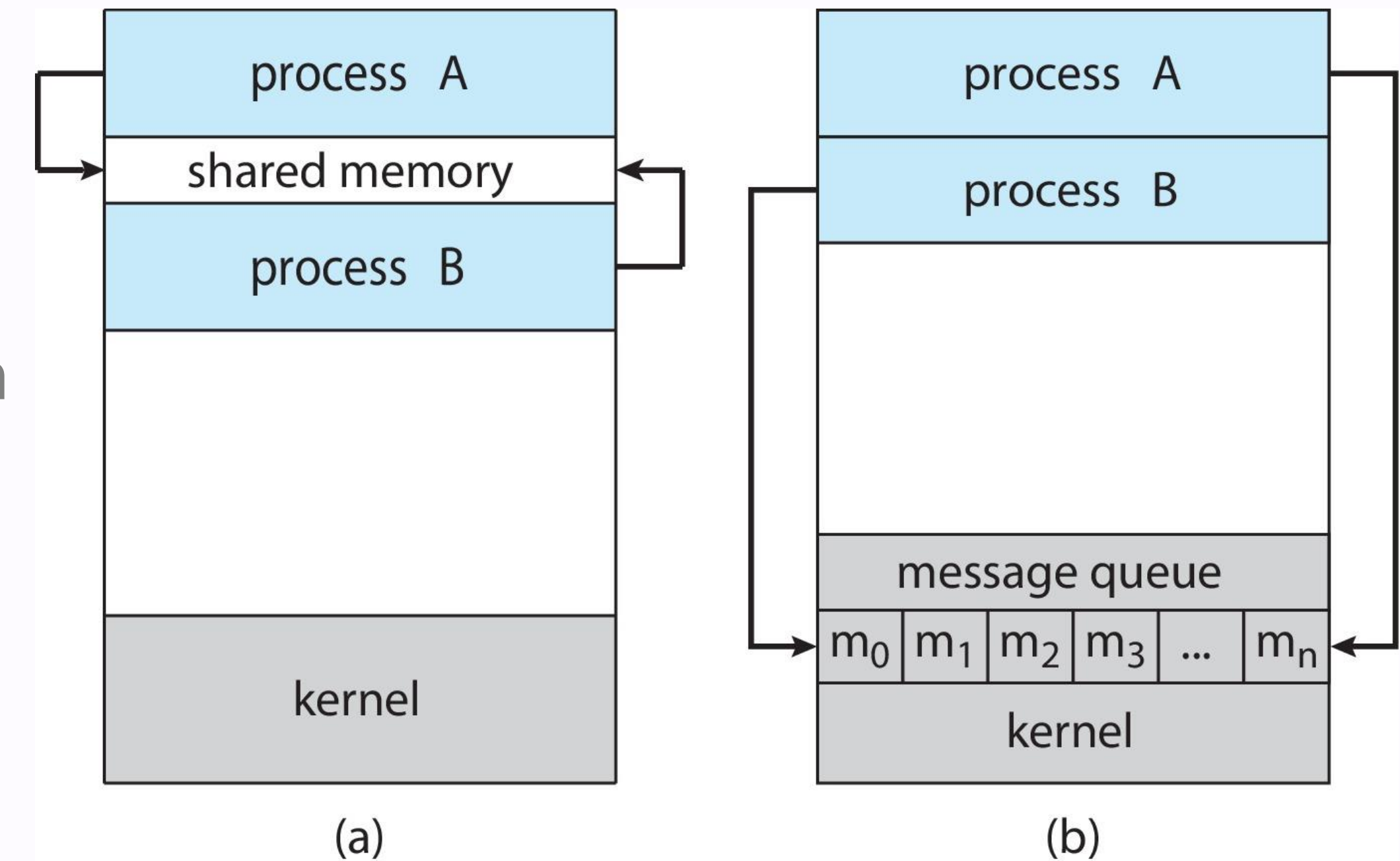
- Compreender a necessidade de IPC
- Distinguir processos independentes vs cooperantes
- Conhecer modelos de IPC:
 - Memória partilhada
 - Passagem de mensagens
- Dominar mecanismos IPC:
 - Pipes, Named Pipes
 - Message queues
 - Sockets
- Explorar comunicação cliente-servidor
- Compreender avanços modernos (2024-2026):
 - Zero-copy e RDMA
 - gRPC e Protocol Buffers
 - Message brokers (Kafka, RabbitMQ)
 - Service mesh e microservices

PROCESSOS COOPERANTES

- Processos independentes:
 - Não afetam nem são afetados por outros
 - Executam isoladamente
- Processos cooperantes:
 - Podem influenciar-se mutuamente
 - Partilham dados ou recursos
- Razões para cooperação:
 - Partilha de informação
 - Aceleração de computação (paralelismo)
 - Modularidade (separação de responsabilidades)
 - Conveniência (reutilização)
- Necessidade de IPC:
 - Trocar dados
 - Sincronizar ações
 - Coordenar atividades

BENEFÍCIOS DAS THREADS

- Dois modelos fundamentais:
 - 1. Memória Partilhada:
 - Processos acedem região de memória comum
 - Rápido (sem cópias de dados)
 - Requer sincronização explícita
 - Exemplo: POSIX shared memory, mmap
 - 2. Passagem de Mensagens:
 - Processos trocam mensagens
 - Não requer variáveis partilhadas
 - Mais fácil de usar em sistemas distribuídos
 - Exemplo: pipes, sockets, message queues
- Trade-offs:
 - Shared memory: mais rápido, mais complexo
 - Message passing: mais simples, overhead de cópia



MEMÓRIA PARTILHADA

- Conceito:
 - Região de memória acessível por múltiplos processos
- Vantagens:
 - Alta performance (sem cópias)
 - Baixa latência
- Desvantagens:
 - Requer sincronização (semáforos, mutexes)
 - Problemas: race conditions, deadlocks
- APIs:
 - POSIX: shm_open, mmap, shm_unlink
 - System V: shmget, shmat, shmdt
- Problema clássico: Produtor-Consumidor
 - Buffer partilhado
 - Produtor adiciona itens
 - Consumidor retira itens
 - Sincronização crítica

PASSAGEM DE MENSAGENS

- Operações básicas:
 - `send(message)` - enviar
 - `receive(message)` - receber
- Questões de implementação:
 - Como estabelecer ligação?
 - Comunicação direta ou indireta?
 - Síncrona ou assíncrona?
 - Buffering automático ou explícito?
- Comunicação direta:
 - `send(P, msg)` - enviar para processo P
 - `receive(Q, msg)` - receber de processo Q
- Comunicação indireta:
 - Mailboxes/portos
 - `send(A, msg)` - enviar para mailbox A
 - `receive(A, msg)` - receber de mailbox A

SINCRONIZAÇÃO EM MESSAGE PASSING

- Envio bloqueante (síncrono):
 - Emissor espera até mensagem ser recebida
- Envio não-bloqueante (assíncrono):
 - Emissor envia e continua
- Receção bloqueante:
 - Recetor espera até mensagem disponível
- Receção não-bloqueante:
 - Recetor obtém mensagem ou null
- Rendezvous:
 - Envio e receção bloqueantes
 - Sincronização explícita
- Buffering:
 - Capacidade zero: rendezvous obrigatório
 - Capacidade limitada: emissor espera se cheio
 - Capacidade ilimitada: emissor nunca espera

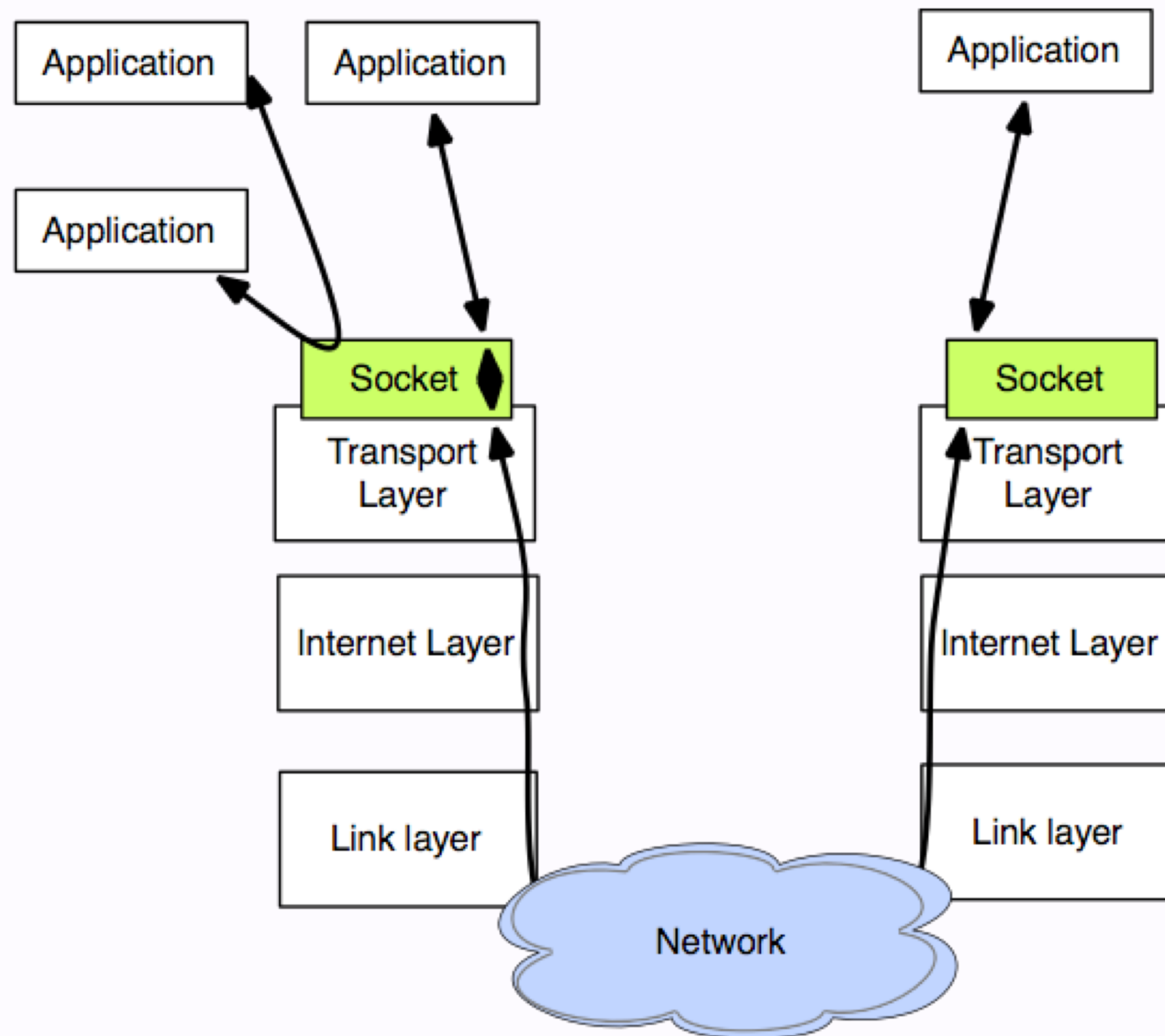
PIPES

- Conceito:
 - Conduta unidirecional para comunicação
 - Dados fluem numa direção
- Pipes Ordinários:
 - Comunicação pai-filho
 - Unidirecionais
 - Não acessíveis fora do processo criador
 - UNIX: pipe(), read(), write()
 - Windows: anonymous pipes
- Named Pipes (FIFOs):
 - Bidirecionais
 - Persistem no filesystem
 - Não requerem relação pai-filho
 - Múltiplos processos podem usar
 - UNIX: mkfifo()
 - Windows: CreateNamedPipe()

SOCKETS

- Definição:
 - Endpoint para comunicação
 - IP address + port number
- Exemplo:
 - 161.25.19.8:80 (web server)
 - 127.0.0.1:8080 (localhost)
- Tipos:
 - Stream sockets (TCP): confiável, ordenado
 - Datagram sockets (UDP): não confiável, rápido
- Portos bem conhecidos:
 - HTTP: 80, HTTPS: 443
 - SSH: 22, FTP: 21
 - <1024: privilegiados
- APIs:
 - `socket()`, `bind()`, `listen()`, `accept()`
 - `connect()`, `send()`, `recv()`, `close()`
- Uso: comunicação em rede, IPC local

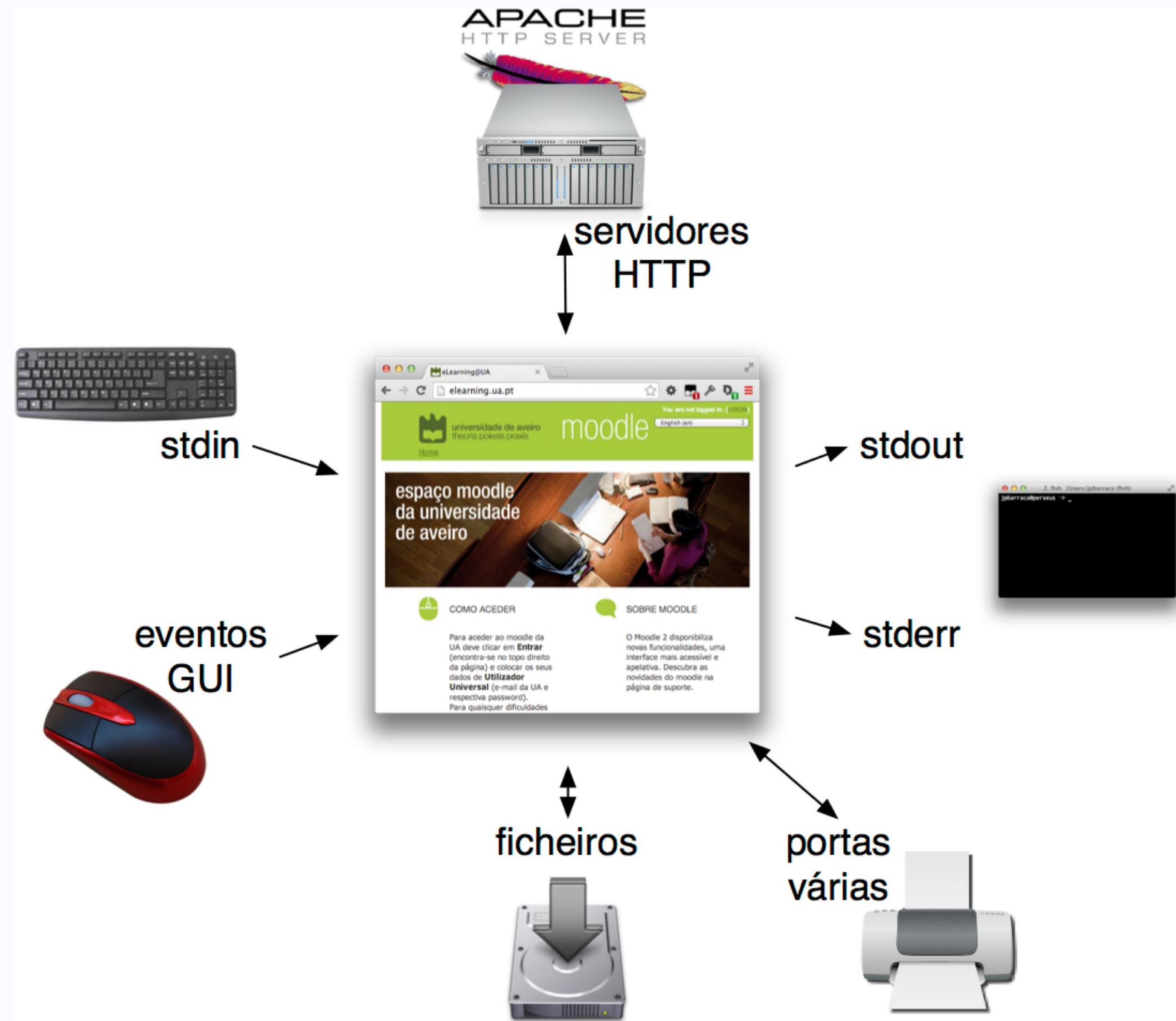
COMUNICAÇÃO ENTRE PROCESSOS - SOCKETS



COMUNICAÇÃO

Aplicações interagem de várias formas

- com o utilizador (stdin, stdout, stderr)
- com dispositivos (portas USB, Serie, etc..)
- Ficheiros
- eventos de interação (apontadores)
- **com outras aplicações**



Exemplo: Navegador Web

COMUNICAÇÃO ENTRE APLICAÇÕES

Mecanismo muito importante!

- Aplicações na Internet

- Aplicações e servidor gráfico (X11)

Implementado de diferentes formas

- PIPE – Redireccionamento através de stdin e stdout

- Socket – Ligação de entre aplicações

- Outros

PIPE

Mecanismo básico

Pode ser transparente para aplicações

Âmbito local (mesma máquina)

Liga stdout/stderr a stdin de aplicações

Exemplo: contar interfaces de rede

```
$ ifconfig -a | grep 'Link encap' | wc -l 4
```

```
$ netstat -nlp
```


SOCKET

Abstração semelhante a uma tomada de parede

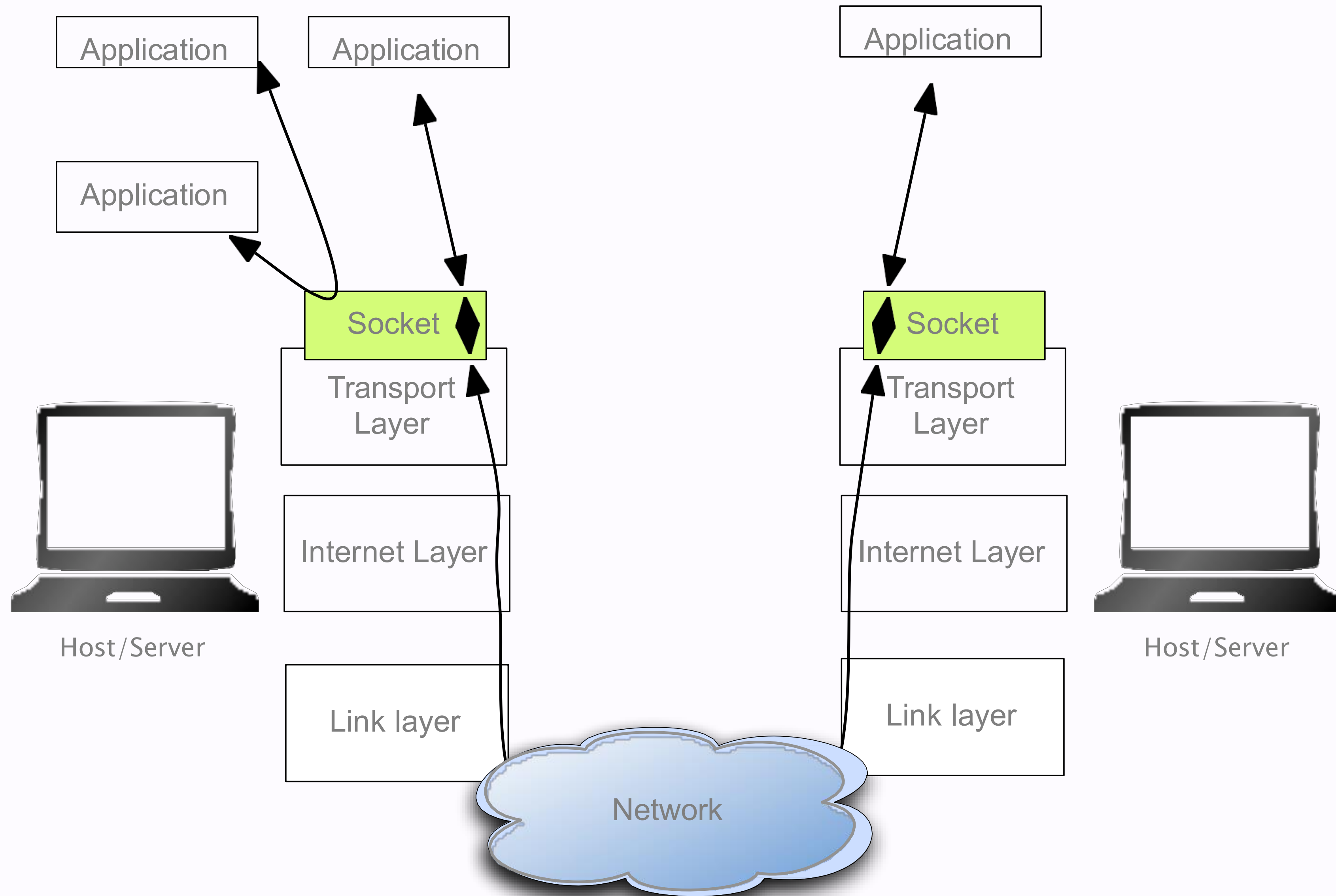
Aplicações definem características

- ficam abertos para comunicação

- tal como uma ficha tem formato e contatos específicos

Comunicação interna ou externa





SOCKET: CARACTERÍSTICAS

1. Família

2. Tipo

3. Nome

Restrição: só 1 socket da mesma família, tipo e nome.

SOCKET: FAMÍLIA

Indica qual o protocolo de rede a usar

AF_UNIX: comunicações locais (sem IP)

AF_INET/AF_INET6: usar IPv4/IPv6

Para comunicações locais ou remotas

... outros

SOCKET: TIPO

Define o protocolo de transporte a usar

SOCK_DGRAM: Não orientado à ligação

Usa UDP

SOCK_STREAM: Orientado à ligação

Usa TCP

SOCKET: NOME

Identifica qual o socket num sistema

Formato depende da família

AF_UNIX: usa nome de ficheiro

Ex: /tmp/run.sock

AF_INET/AF_INET6: usa endereço e porta

Um interface: 127.0.0.1:1234

Todos os interfaces: 0.0.0.0:1234

MODELO CLIENTE-SERVIDOR

Sockets assumem que existe separação de funções

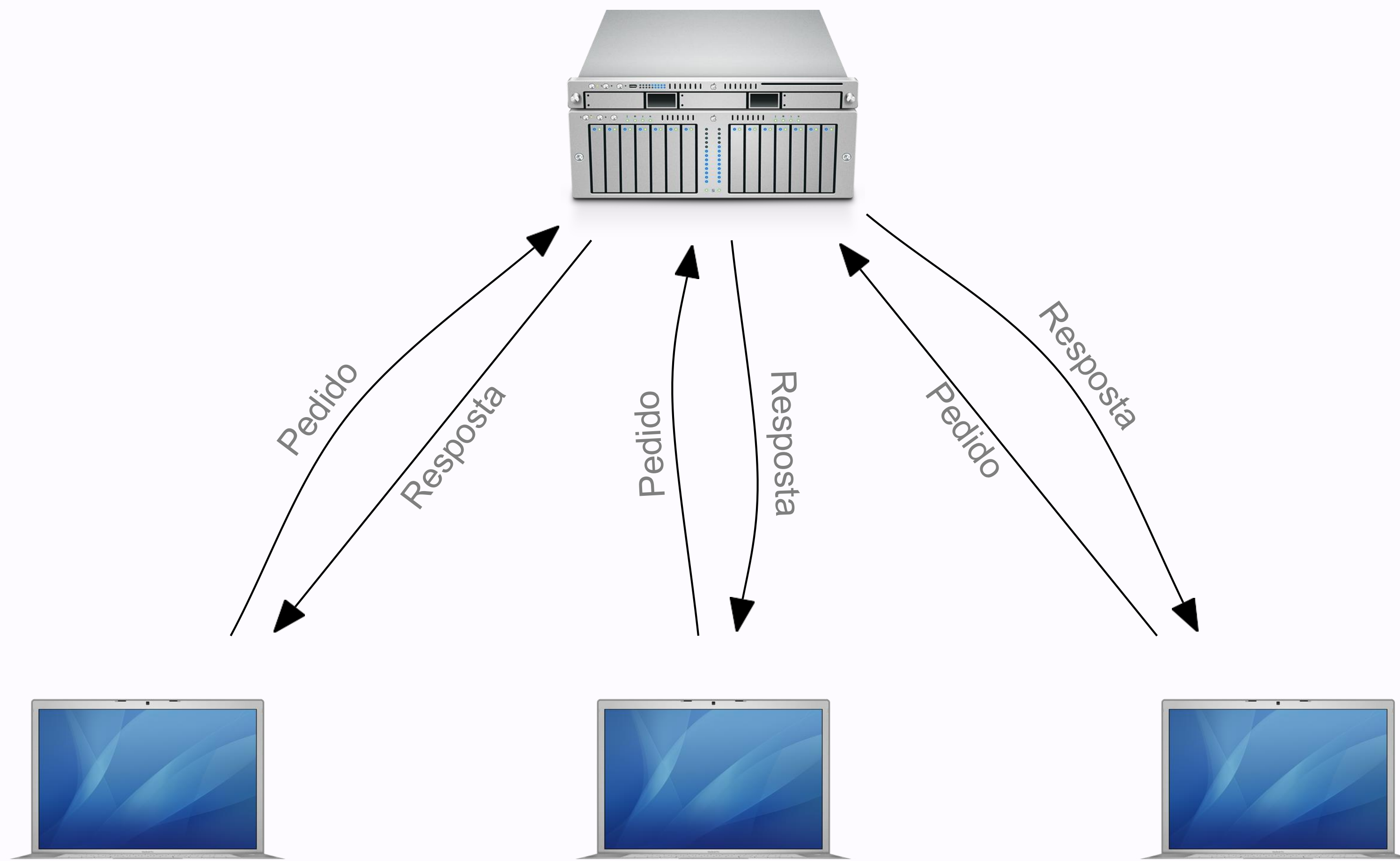
Cliente: efetua pedidos

Ex: navegador

Servidor: responde a pedidos

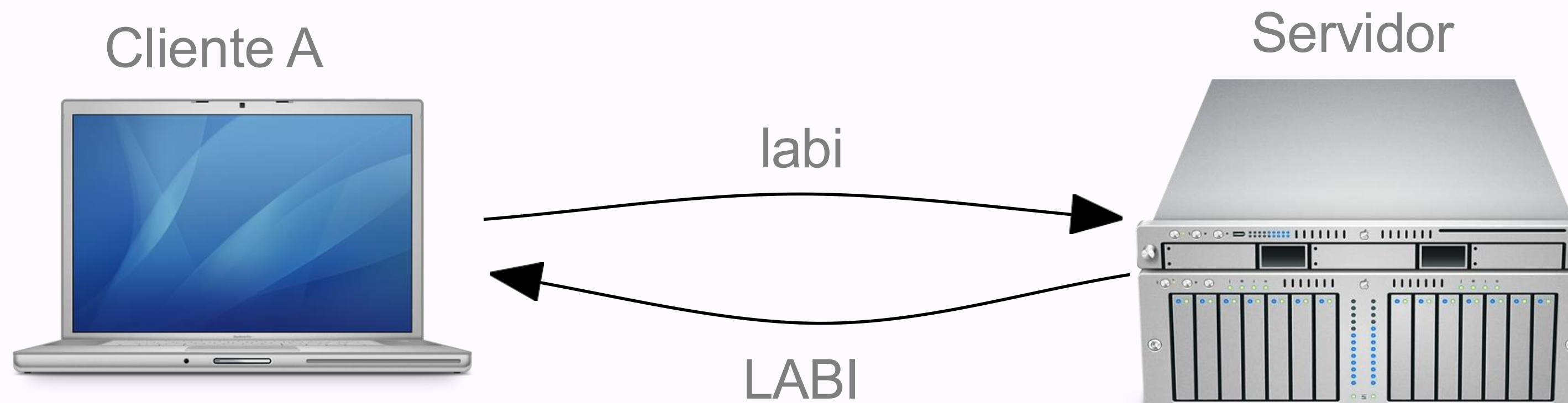
Ex: servidor HTTP

MODELO CLIENTE-SERVIDOR

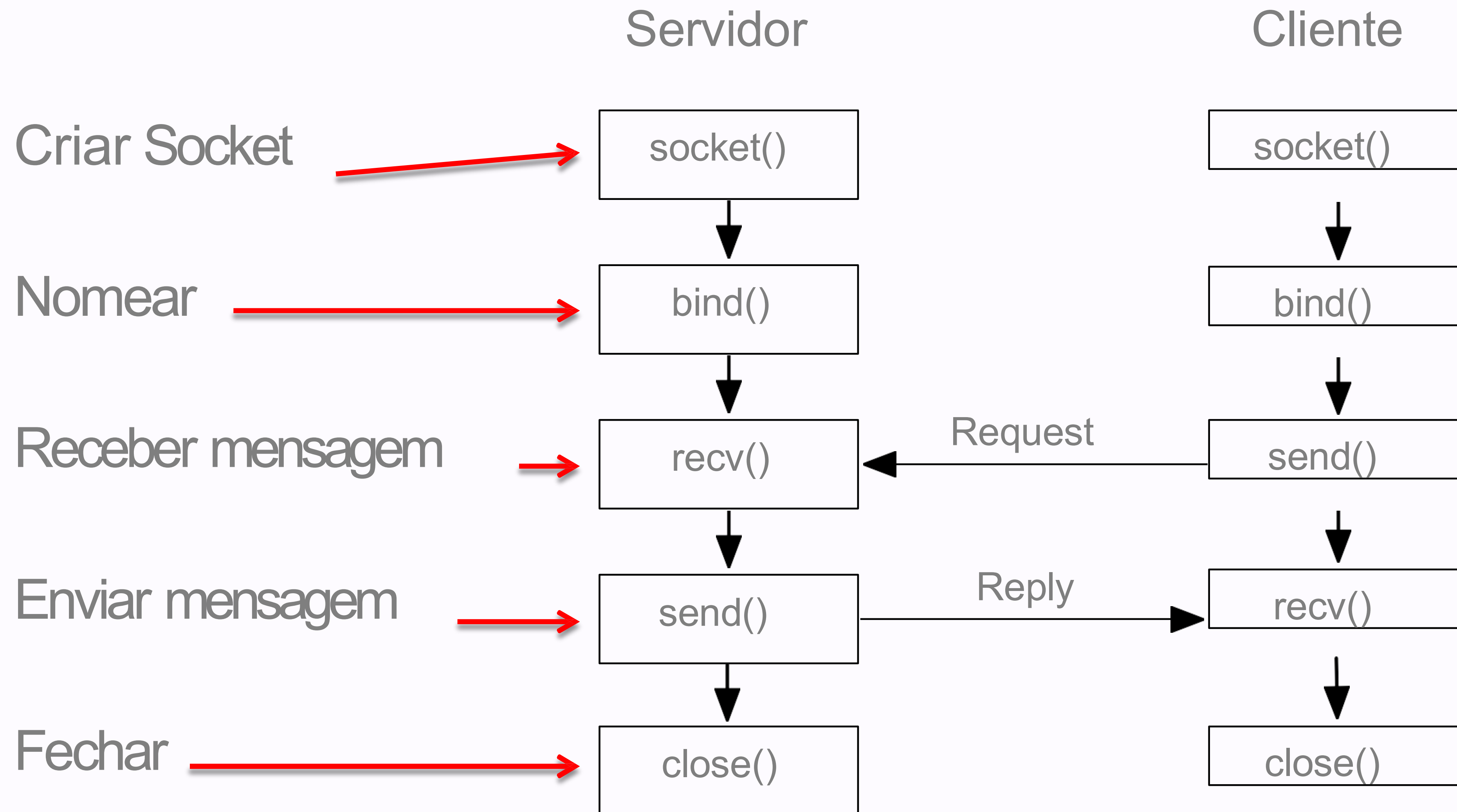


EXEMPLO: UDP ECHO SERVER

- ❑ Cliente: envia mensagem para servidor
- ❑ Servidor: responde com mensagem em maiúsculas



AÇÕES: UDP ECHO



UDP: CRIAR SOCKETS (SERVIDOR)

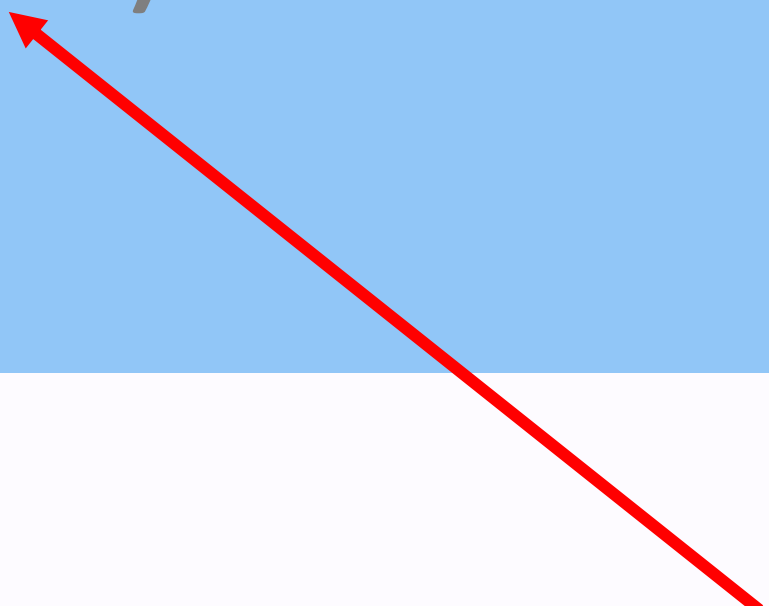
```
from socket import *
```

Importar módulo



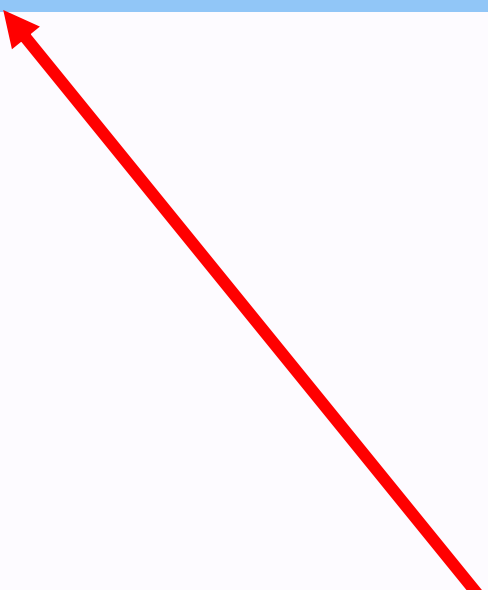
```
udp_s = socket(AF_INET, SOCK_DGRAM)
```

Criar socket



```
udp_s.bind( ('0.0.0.0',1234) )
```

Nomear Socket
(todos os interfaces, porta 1234)



UDP: TRANSMITIR INFORMAÇÃO (SERVIDOR)

Dados recebidos

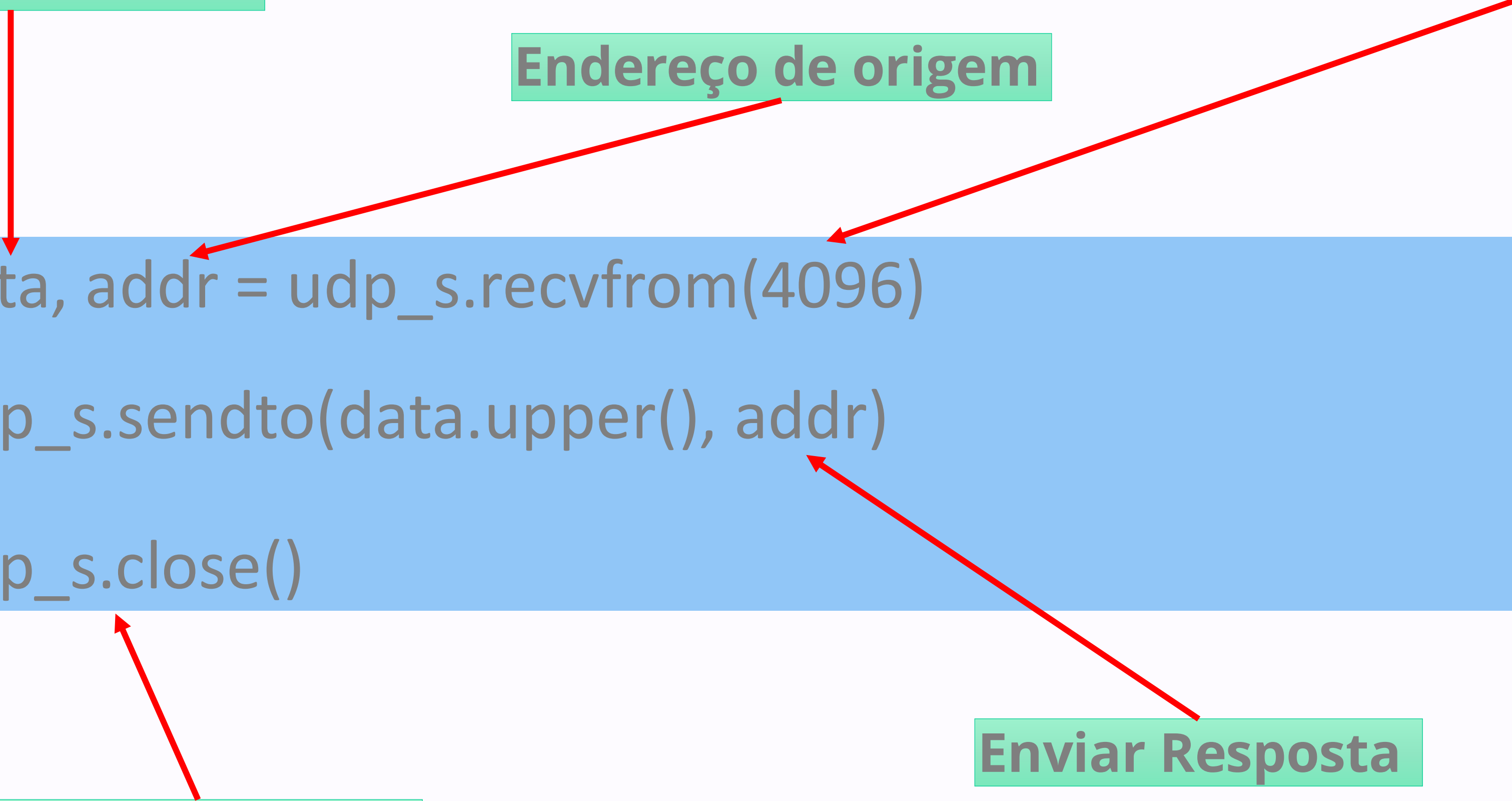
Receber mensagem até 4096 bytes

Endereço de origem

```
data, addr = udp_s.recvfrom(4096)  
udp_s.sendto(data.upper(), addr)  
udp_s.close()
```

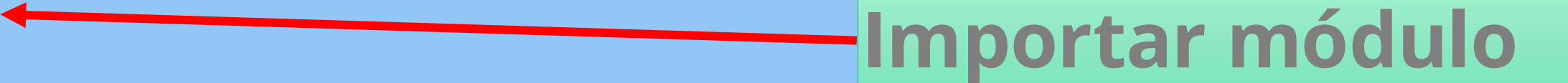
Enviar Resposta

Fechar Socket



UDP: CRIAR SOCKETS (CLIENTE)

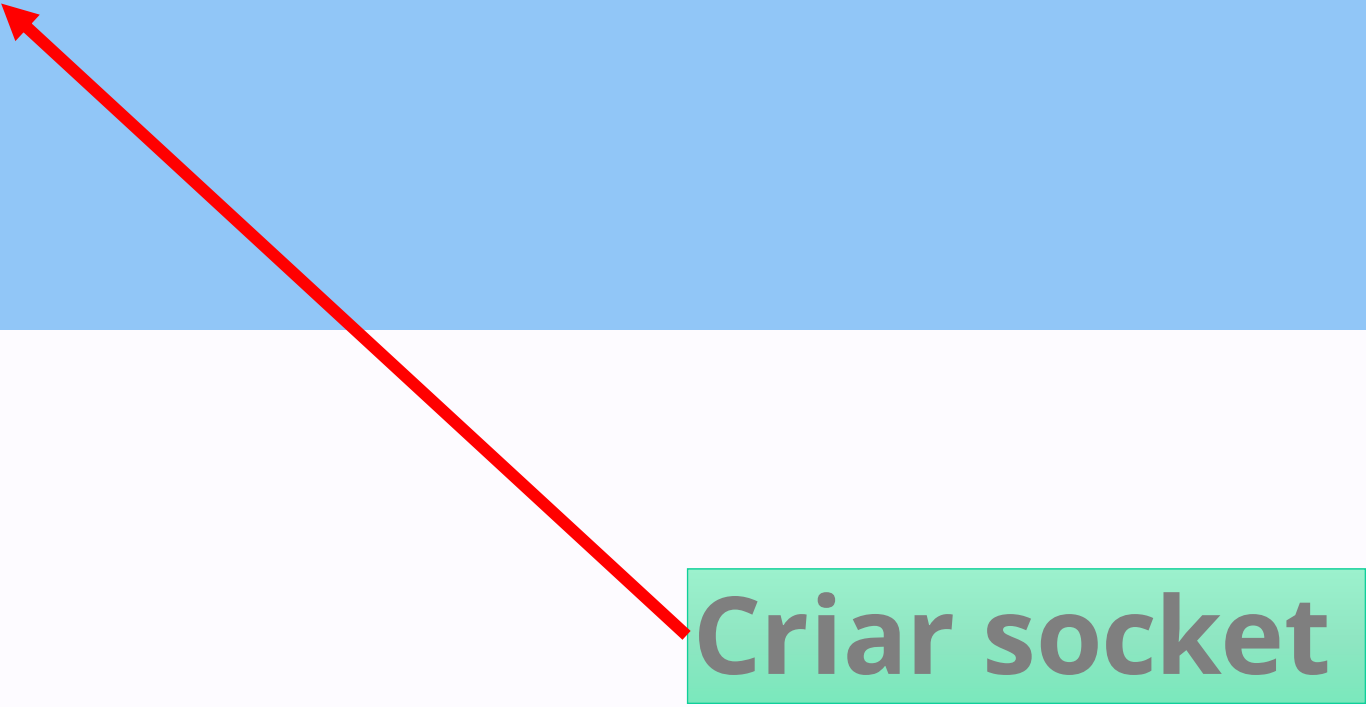
```
from socket import *
```



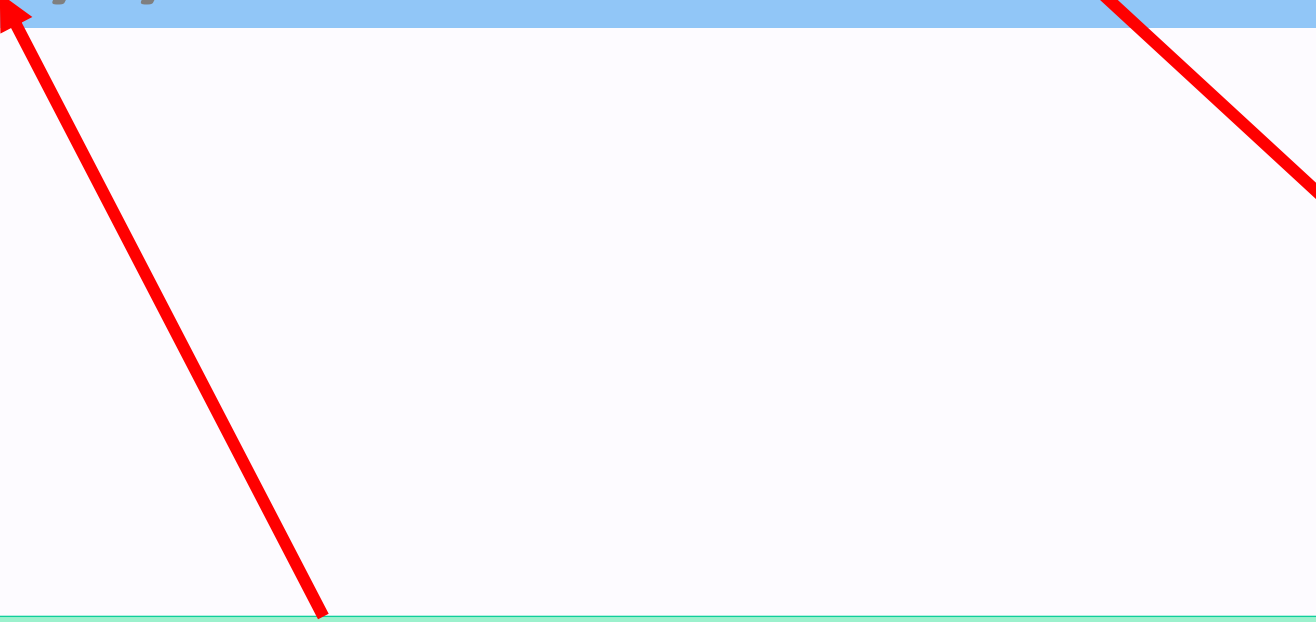
Importar módulo

```
udp_s = socket(AF_INET, SOCK_DGRAM)
```

```
udp_s.bind( ('0.0.0.0',0) )
```



Criar socket



Nomear Socket
(todos os interfaces, porta **aleatória**)

UDP: TRANSMITIR INFORMAÇÃO (CLIENTE)

Dados recebidos

Endereço de origem

Enviar mensagem

```
udp_s.sendto('labi'.encode('utf-8'), ('127.0.0.1',1234) )  
data, addr = udp_s.recvfrom(4096)  
print data  
udp_s.close()
```

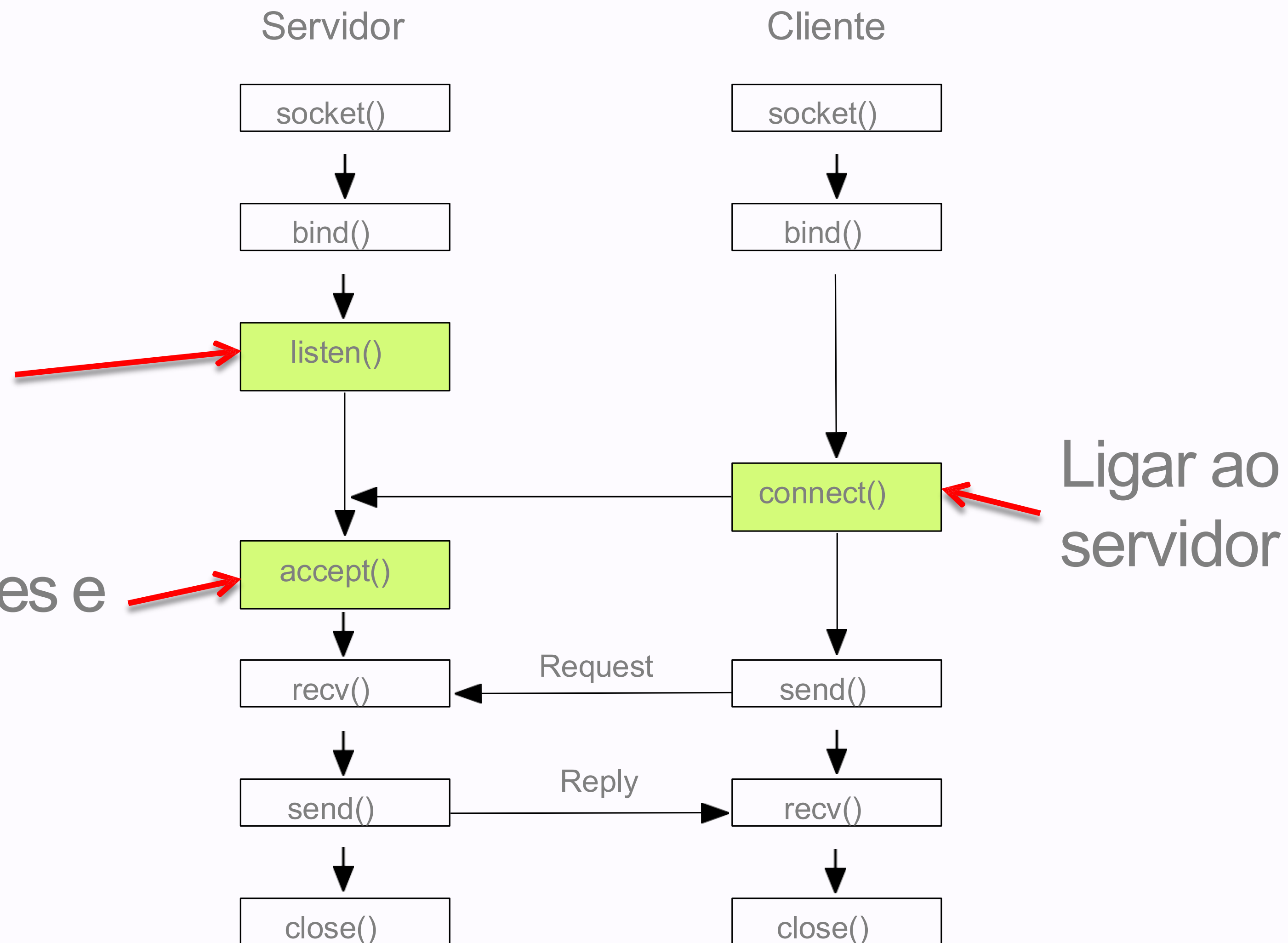
Enviar Resposta

Fechar Socket

AÇÕES: TCP

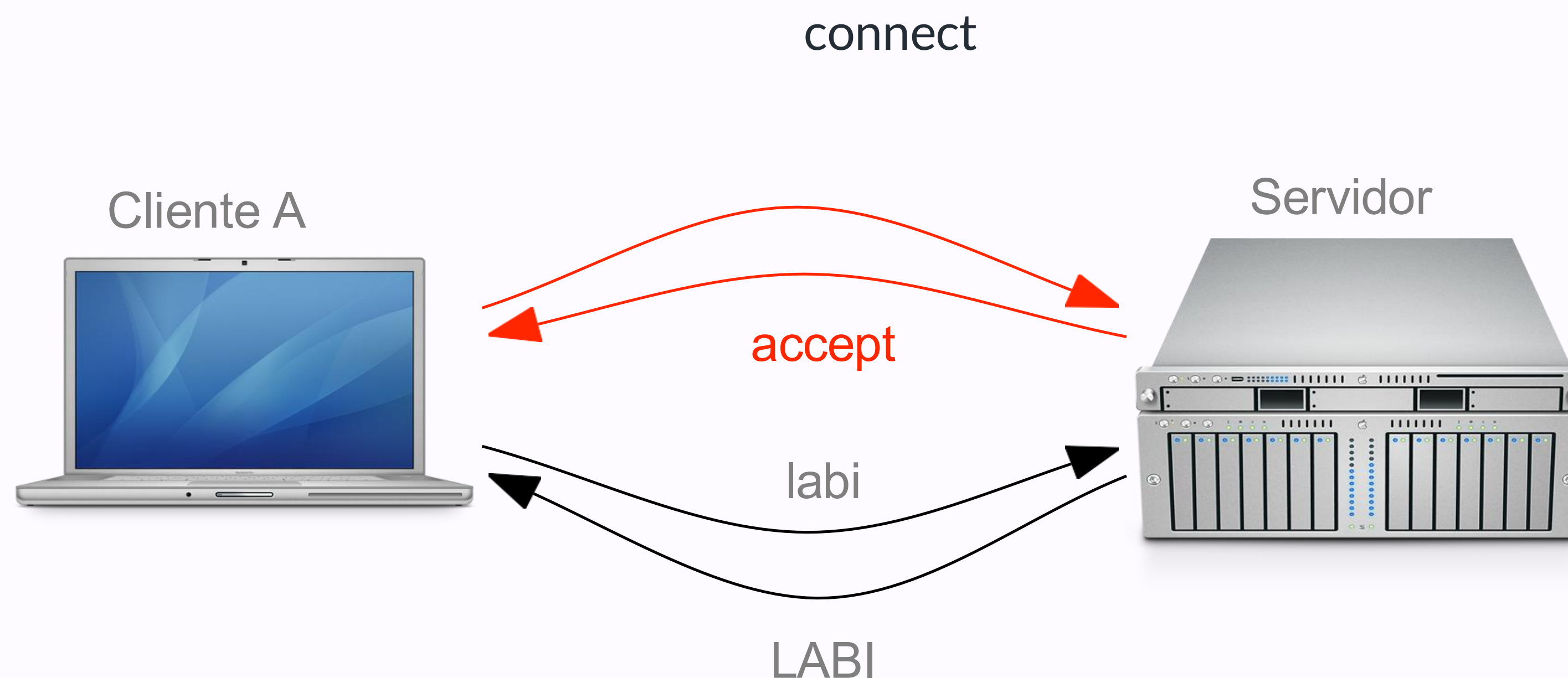
Socket vai aceitar ligações

Esperar por ligações e aceitá-las



EXEMPLO: TCP ECHO SERVER

- Cliente: estabelece ligação e envia mensagem para servidor
- Servidor: aceita ligação e responde com mensagem em maiúsculas



TCP: CRIAR SOCKETS (SERVIDOR)

Criar socket TCP

...

```
tcp_s = socket(AF_INET, SOCK_STREAM)
```

```
tcp_s.bind( ('0.0.0.0',1234) )
```

```
tcp_s.listen(1)
```

```
client_s, addr = tcp_s.accept()
```

Aceitar
ligações

Criado novo Socket
Identifica a ligação

Nomear Socket
(todos os interfaces, porta 1234)

TCP: TRANSMITIR INFORMAÇÃO (SERVIDOR)

Dados recebidos

Receber mensagem
até 4096 bytes

```
data = client_s.recv(4096)
client_s.send(data.upper())
client_s.close()
tcp_s.close()
```

Enviar Resposta

client_s identifica univocamente ligação servidor <-> cliente

TCP: CRIAR SOCKETS (CLIENTE)

Importar módulo

```
from socket import *
```

```
tcp_s = socket(AF_INET, SOCK_STREAM)
```

```
tcp_s.bind( ('0.0.0.0',0) )
```

```
tcp_s.connect( ('127.0.0.1', 1234))
```

Criar Socket

Ligar ao servidor

TCP: TRANSMITIR INFORMAÇÃO (CLIENTE)

Dados recebidos

Enviar Mensagem

```
tcp_s.send(labi.encode("utf-8"))  
data = tcp_s.recv(4096)  
print(data)  
tcp_s.close()
```

Receber mensagem
até 4096 bytes

tcp_s identifica ligação cliente<->servidor

ZERO-COPY IPC (2024-2026)

- Conceito:
 - Evitar cópias de dados entre processos
 - Passar ponteiros em vez de copiar
- Vantagens:
 - Redução drástica de latência
 - Menor uso de CPU
 - Maior throughput
- Resultados 2024-2026:
 - RPCool (CXL shared memory): 1.93× vs RDMA, 7.2× vs CXL RPC
 - ROS 2 Agnocast: true zero-copy pub/sub
 - Shared-memory interposition: ordem de magnitude vs loopback
- Desafios:
 - Proteção de memória (MPK - Memory Protection Keys)
 - Sincronização
- Tecnologias:
 - CXL (Compute Express Link)
 - MPKLink (intra-container)

RDMA (2024-2026)

- Remote Direct Memory Access:
 - Acesso direto à memória remota
 - Bypass do kernel e CPU remoto
- Vantagens:
 - Latência ultra-baixa (microsegundos)
 - Alto throughput
 - Baixo uso de CPU
- Resultados 2024-2026:
 - RR-Compound (gRPC+RDMA):
 - 2.35× throughput
 - 47% redução latência
 - RB² (middle layer): 2.5-7.5× throughput
 - OHIO (Alltoall): 37-61% latency reduction
 - Hyperscale AI: >15% network improvement
- Aplicações:
 - HPC, ML training, storage
- Tecnologias: InfiniBand, RoCE, iWARP

gRPC e Protocol Buffers (2024-2026)

- gRPC:
 - Framework RPC moderno (Google)
 - HTTP/2, bidirectional streaming
 - Multi-linguagem (C++, Java, Python, Go, etc.)
- Protocol Buffers:
 - Formato de serialização binário
 - Compacto, rápido, type-safe
- Vantagens vs REST:
 - Menor payload (binário vs JSON)
 - Streaming bidirecional
 - Type safety
- Resultados 2024-2026:
 - gRPC+RDMA: 2.35× throughput, 47% latência
 - Protobuf offload (DPU): redução significativa CPU
 - vs REST: depende do workload
- Desvantagens:
 - Mais complexo que REST
 - Debugging mais difícil

MESSAGE BROKERS

- Conceito:
 - Middleware para passagem de mensagens
 - Desacopla produtores e consumidores
- Apache Kafka:
 - Alto throughput, persistência
 - Streaming de eventos
 - Usado: LinkedIn, Netflix, Uber
- RabbitMQ:
 - AMQP protocol
 - Routing flexível
 - Garantias de entrega
- NATS:
 - Lightweight, alta performance
 - Cloud-native
- Redis Streams:
 - In-memory, baixa latência
- Trade-offs (2024-2026):
 - Kafka: throughput vs latência
 - RabbitMQ: flexibilidade vs complexidade

SERVICE MESH (2024-2026)

- Conceito:
 - Infraestrutura para comunicação entre microservices
 - Proxy sidecar em cada serviço
- Funcionalidades:
 - Load balancing
 - Service discovery
 - Circuit breaking
 - Observability (metrics, tracing)
 - Security (mTLS, autenticação)
- Principais implementações:
 - Istio: completo, Kubernetes-native
 - Linkerd: lightweight, Rust
 - Envoy: proxy de alto desempenho
- Overhead:
 - Latência adicional (proxy)
 - Memória (sidecar por pod)
- Estado 2024-2026:
 - Adoção crescente em cloud-native
 - Otimizações de performance

MICROSERVICES COMMUNICATION

- Padrões de comunicação:
 - Síncrono: REST, gRPC
 - Assíncrono: message queues, event streaming
- Desafios:
 - Network latency
 - Partial failures
 - Distributed tracing
 - Service discovery
- Best practices:
 - Circuit breakers (evitar cascading failures)
 - Retry com backoff exponencial
 - Timeouts adequados
 - Idempotência
- Otimizações 2024-2026:
 - RDMA para low-latency
 - Shared memory para co-located services
 - HTTP/3 (QUIC) para melhor performance
- Observability: OpenTelemetry, Jaeger

QUIC e HTTP/3

- QUIC (Quick UDP Internet Connections):
 - Protocolo de transporte sobre UDP
 - Desenvolvido por Google
- Vantagens vs TCP:
 - Conexão mais rápida (0-RTT)
 - Sem head-of-line blocking
 - Migração de conexão (mobile)
 - Encriptação por defeito (TLS 1.3)
- HTTP/3:
 - HTTP sobre QUIC
 - Sucessor de HTTP/2
- Benefícios 2024-2026:
 - Melhor performance em redes instáveis
 - Redução de latência em handshake
 - Multiplexing sem bloqueio
- Adoção:
 - Google, Facebook, Cloudflare
 - Suporte crescente em browsers e servers

WEBSOCKETS E SERVER-SENT EVENTS

- WebSockets:
 - Comunicação bidirecional full-duplex
 - Sobre TCP, upgrade de HTTP
 - Baixa latência, real-time
 - Casos de uso: chat, gaming, live updates
- Server-Sent Events (SSE):
 - Comunicação unidirecional (server→client)
 - Sobre HTTP
 - Mais simples que WebSockets
 - Casos de uso: notifications, live feeds
- Comparação:
 - WebSockets: bidirecional, mais complexo
 - SSE: unidirecional, mais simples
- Alternativas modernas:
 - gRPC streaming
 - HTTP/3 server push
- Performance: WebSockets mais eficiente para bidirecional

CONTAINER NETWORKING

- Desafios:
 - Isolamento de rede entre containers
 - Service discovery
 - Load balancing
- Docker networking:
 - Bridge: rede privada por host
 - Host: usa rede do host
 - Overlay: multi-host (Swarm)
- Kubernetes networking:
 - Pod network: cada pod tem IP
 - Service: abstração de endpoints
 - Ingress: entrada HTTP/HTTPS
- CNI (Container Network Interface):
 - Calico, Flannel, Weave, Cilium
- Otimizações 2024-2026:
 - eBPF para networking (Cilium)
 - RDMA para low-latency
 - Shared memory para co-located pods

eBPF para Networking

- eBPF (Extended Berkeley Packet Filter):
 - Programas no kernel Linux
 - Seguro, verificado, JIT-compiled
- Casos de uso em networking:
 - Packet filtering
 - Load balancing
 - Network observability
 - Security (DDoS protection)
- Cilium:
 - CNI baseado em eBPF
 - Alta performance
 - Service mesh capabilities
- Vantagens:
 - Sem modificação do kernel
 - Baixo overhead
 - Programável
- Aplicações 2024-2026:
 - Kubernetes networking
 - Service mesh data plane
 - Cloud-native observability

UNIX DOMAIN SOCKETS

- Conceito:
 - Sockets para IPC local (mesmo host)
 - Mais rápidos que TCP loopback
- Vantagens:
 - Menor latência
 - Maior throughput
 - Sem overhead de rede
 - Pode passar file descriptors
- API:
 - `socket(AF_UNIX, SOCK_STREAM, 0)`
 - `bind()` a um path no filesystem
- Performance vs TCP loopback:
 - 2-3× mais rápido
 - Menos context switches
- Casos de uso:
 - Docker daemon
 - Database connections (PostgreSQL)
 - X11 display server
- Best practice: Preferir para IPC local

MESSAGE QUEUES

- Conceito:
 - Filas de mensagens persistentes
 - Comunicação assíncrona
- POSIX Message Queues:
 - `mq_open`, `mq_send`, `mq_receive`
 - Prioridades de mensagens
 - Notificação assíncrona
- System V Message Queues:
 - `msgget`, `msgsnd`, `msgrcv`
 - Mais antigo, menos features
- Vantagens:
 - Desacoplamento temporal
 - Buffering automático
 - Persistência
- Desvantagens:
 - Overhead de cópia
 - Complexidade de gestão
- Modernos: Kafka, RabbitMQ, NATS
 - Mais features, escalabilidade

RPC (REMOTE PROCEDURE CALL)

- Conceito:
 - Chamar função em processo remoto
 - Abstração de comunicação distribuída
- Componentes:
 - Stub cliente: marshalling
 - Stub servidor: unmarshalling
 - RPC runtime: transporte
- Desafios:
 - Falhas de rede
 - Latência
 - Heterogeneidade
- Frameworks clássicos:
 - Sun RPC, DCE RPC
- Frameworks modernos:
 - gRPC (Google)
 - Apache Thrift (Facebook)
 - Cap'n Proto
- Performance 2024-2026:
 - gRPC+RDMA: 2.35× throughput
 - Protobuf offload: redução CPU

SINCRONIZAÇÃO E COORDENAÇÃO

- Necessidade:
 - Evitar race conditions
 - Garantir consistência
- Primitivas de sincronização:
 - Semáforos: contadores
 - Mutexes: exclusão mútua
 - Condition variables: wait/signal
 - Read-write locks: múltiplos leitores
- Coordenação distribuída:
 - Consensus: Paxos, Raft
 - Leader election
 - Distributed locks
- Serviços de coordenação:
 - ZooKeeper: configuração, naming, sync
 - etcd: key-value store (Kubernetes)
 - Consul: service mesh, discovery
- Best practices:
 - Minimizar secções críticas
 - Evitar deadlocks

PERFORMANCE E TRADE-OFFS

- Latência:
 - Shared memory < Unix sockets < TCP loopback < Network
- Throughput:
 - RDMA > Shared memory > Sockets
- Complexidade:
 - Sockets < Message queues < Shared memory
- Escalabilidade:
 - Message passing > Shared memory
- Resultados 2024-2026:
 - Zero-copy: 1.93-7.2× speedup
 - RDMA: 2.35× throughput, 47% latência
 - gRPC: depende do workload
- Escolha do mecanismo:
 - Local + performance: shared memory
 - Local + simplicidade: Unix sockets
 - Distribuído: gRPC, message brokers
 - HPC: RDMA

BEST PRACTICES

- Design:
 - Escolher mecanismo adequado ao caso de uso
 - Considerar latência, throughput, complexidade
- Performance:
 - Zero-copy quando possível
 - RDMA para HPC/ML
 - Unix sockets para IPC local
- Confiabilidade:
 - Timeout e retry com backoff
 - Circuit breakers
 - Idempotência
- Segurança:
 - Autenticação e autorização
 - Encriptação (TLS)
 - Validação de inputs
- Observability:
 - Logging estruturado
 - Distributed tracing
 - Metrics (latência, throughput, erros)

TENDÊNCIAS FUTURAS

- Zero-copy e RDMA:
 - Adoção crescente
 - CXL para shared memory
- Protocolos modernos:
 - HTTP/3 (QUIC) dominante
 - gRPC para microservices
- Service mesh:
 - Padrão em cloud-native
 - eBPF data plane

TENDÊNCIAS FUTURAS

- Edge computing:
 - IPC otimizado para IoT
 - Low-latency critical
- AI/ML:
 - RDMA para distributed training
 - Collective communication optimizations
- Hardware:
 - SmartNICs, DPUs
 - Offload de processamento
- Segurança:
 - Zero-trust networking
 - Confidential computing

MÉTRICAS 2024-2026

- Zero-Copy:
 - RPCool: 1.93× vs RDMA, 7.2× vs CXL RPC
 - Shared-memory: ordem de magnitude vs loopback
- RDMA:
 - RR-Compound: 2.35× throughput, 47% latência
 - RB²: 2.5-7.5× throughput
 - OHIO: 37-61% latency reduction
 - Hyperscale AI: >15% network improvement
- gRPC:
 - gRPC+RDMA: 2.35× throughput
 - Protobuf offload: redução significativa CPU
- Message Brokers:
 - Kafka: alto throughput
 - RabbitMQ: flexibilidade
- Service Mesh:
 - Overhead: latência adicional
 - Trade-off: features vs performance

CONCLUSÕES

- IPC essencial para processos cooperantes
- Dois modelos: memória partilhada, passagem mensagens
- Mecanismos clássicos: pipes, sockets, message queues
- Modernos (2024-2026):
 - Zero-copy: 1.93-7.2× speedup
 - RDMA: 2.35× throughput, 47% latência
 - gRPC: padrão para microservices
 - Message brokers: Kafka, RabbitMQ, NATS
 - Service mesh: Istio, Linkerd, Envoy
 - HTTP/3 (QUIC): próxima geração
- Trade-offs: latência, throughput, complexidade
- Escolha depende do caso de uso
- Futuro: zero-copy, RDMA, eBPF, edge

QUESTÕES?